

Tái hiện GPT: Xây dựng lại nguyên lí của các mô hình ngôn ngữ lớn hiện tại

Thành viên nhóm

- Nguyễn Văn An - 21010647 - CNTT2
- Bùi Đại Dương - 21012050 - CNTT3
- Bùi Việt Quang - 21010614 - CNTT1

1. Giới thiệu về nanoGPT

NanoGPT, một dự án mã nguồn mở được phát triển bởi Andrej Karpathy, là một triển khai thu nhỏ và đơn giản hóa của mô hình Generative Pre-trained Transformer (GPT). Nó được thiết kế như một công cụ giáo dục và nghiên cứu, giúp người học dễ dàng tiếp cận và hiểu sâu hơn về kiến trúc Transformer, nền tảng của các mô hình ngôn ngữ lớn (LLMs) hiện đại. NanoGPT không chỉ là một mô hình nhỏ; nó là một lộ trình để khám phá và tái hiện nguyên lý cốt lõi của GPT.

2. Bối cảnh và Mục tiêu

Các mô hình GPT lớn (ví dụ: GPT-2, GPT-3) với hàng tỷ tham số đã chứng minh khả năng vượt trội trong nhiều tác vụ xử lý ngôn ngữ tự nhiên (NLP). Tuy nhiên, độ phức tạp và yêu cầu tài nguyên tính toán lớn của chúng gây khó khăn cho việc học tập và thử nghiệm. NanoGPT ra đời để giải quyết vấn đề này bằng cách:

- **Đơn giản hóa Kiến trúc:** Cung cấp một phiên bản GPT với số lượng tham số nhỏ hơn đáng kể, giúp giảm yêu cầu về tài nguyên tính toán và thời gian huấn luyện.
- **Tối ưu hóa cho Giáo dục:** Thiết kế mã nguồn dễ đọc, dễ hiểu và có tài liệu đầy đủ, tạo điều kiện thuận lợi cho việc học tập và thực hành.
- **Tập trung vào Nguyên lý:** Tái hiện các thành phần cốt lõi của kiến trúc Transformer một cách rõ ràng, giúp người học nắm bắt nguyên lý hoạt động của mô hình.

Mục tiêu cụ thể:

- Triển khai mô hình GPT cơ bản từ đầu bằng PyTorch (train.py và model.py, mỗi file khoảng 300 dòng).
- Huấn luyện trên các tập dữ liệu nhỏ (ví dụ: tác phẩm của Shakespeare) để minh họa quá trình học và tạo văn bản.
- Đảm bảo mã nguồn có tài liệu rõ ràng và dễ sửa đổi.
- Chạy được trên phần cứng thông thường (CPU hoặc GPU cấp thấp).
- Có khả năng tùy chỉnh cho các nhiệm vụ NLP khác nhau.

3. Yêu cầu và Hiệu quả

Để đạt được các mục tiêu trên, nanoGPT cần đáp ứng các yêu cầu sau:

- **Đơn giản và Rõ ràng:** Mã nguồn phải dễ hiểu, với tài liệu chi tiết, phù hợp cho người mới bắt đầu.
- **Hiệu quả:** Mô hình phải chạy được trên phần cứng thông thường, đảm bảo tiếp cận được với nhiều người.
- **Linh hoạt:** Có thể tùy chỉnh cho các nhiệm vụ NLP khác nhau, như phân loại văn bản hoặc hỏi đáp.

Hiệu quả của nanoGPT thể hiện ở khả năng tái hiện các đặc tính cơ bản của GPT, mặc dù với quy mô nhỏ hơn. Ví dụ, nó có thể được huấn luyện trên tập dữ liệu nhỏ và tạo ra văn bản có tính liên kết và ngữ nghĩa.

4. Kiến trúc Transformer trong nanoGPT: Chi tiết

NanoGPT tái hiện kiến trúc Transformer thông qua các thành phần chính sau:

- **Embedding Layer:** Chuyển đổi các token đầu vào thành các vector số (embedding). Kích thước embedding (embedding dimension) quyết định khả năng biểu diễn ngữ nghĩa của mô hình. Ví dụ, nếu chúng ta có một từ vựng gồm 10000 từ, mỗi từ sẽ được ánh xạ thành một vector có kích thước, ví dụ, 512.

```
import torch
import torch.nn as nn

class Embedding(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)

    def forward(self, x):
        return self.embedding(x)

# Ví dụ
vocab_size = 10000
embedding_dim = 512
embedding_layer = Embedding(vocab_size, embedding_dim)
input_tensor = torch.randint(0, vocab_size, (32,)) # Batch size 32
embedded_output = embedding_layer(input_tensor)
print(embedded_output.shape) # Output: torch.Size([32, 512])
```

- **Self-Attention Mechanism:** Cho phép mô hình tập trung vào các phần khác nhau của chuỗi đầu vào khi xử lý mỗi token. Quá trình này bao gồm:
 - Tính toán Query (Q), Key (K), và Value (V) từ embedding của các token.
 - Tính Attention Weights bằng cách áp dụng hàm softmax lên tích của Q và K.
 - Tính Attention Output bằng cách nhân Attention Weights với V.

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class SelfAttention(nn.Module):
    def __init__(self, embedding_dim, num_heads):
        super().__init__()
        self.embedding_dim = embedding_dim
        self.num_heads = num_heads
        self.head_dim = embedding_dim // num_heads

        self.query = nn.Linear(embedding_dim, embedding_dim)
        self.key = nn.Linear(embedding_dim, embedding_dim)
        self.value = nn.Linear(embedding_dim, embedding_dim)

    def forward(self, x):
        batch_size, seq_len, embedding_dim = x.size()

        q = self.query(x).view(batch_size, seq_len, self.num_heads,
self.head_dim).transpose(1, 2)
        k = self.key(x).view(batch_size, seq_len, self.num_heads,
self.head_dim).transpose(1, 2)
        v = self.value(x).view(batch_size, seq_len, self.num_heads,
self.head_dim).transpose(1, 2)

        attention_weights = torch.softmax(torch.matmul(q,
k.transpose(-2, -1)) / (self.head_dim ** 0.5), dim=-1)
        attention_output = torch.matmul(attention_weights, v)

        attention_output = attention_output.transpose(1,
2).contiguous().view(batch_size, seq_len, embedding_dim)
        return attention_output

# Ví dụ
batch_size = 32
seq_len = 50
embedding_dim = 512
num_heads = 8

self_attention_layer = SelfAttention(embedding_dim, num_heads)
input_tensor = torch.randn(batch_size, seq_len, embedding_dim)
attention_output = self_attention_layer(input_tensor)
print(attention_output.shape) # Output: torch.Size([32, 50, 512])

```

- **Multi-Head Attention:** Sử dụng nhiều "head" attention để mô hình có thể học các mối quan hệ khác nhau giữa các token. Các head attention khác nhau hoạt động song song và sau đó được kết hợp lại.
- **Feedforward Network:** Áp dụng một mạng neural hai lớp cho mỗi vị trí trong chuỗi sau lớp self-attention, thêm tính phi tuyến vào mô hình.

- **Layer Normalization and Residual Connections:** Layer normalization giúp ổn định quá trình huấn luyện, và residual connections (skip connections) giúp ngăn chặn vấn đề vanishing gradient.
- **Decoder-Only Architecture:** NanoGPT sử dụng kiến trúc decoder-only, tập trung vào việc tạo ra chuỗi đầu ra dựa trên chuỗi đầu vào đã cho.
- **Số lượng Layer và Kích thước Head:** Số lượng lớp Transformer và kích thước của mỗi attention head được sử dụng trong nanoGPT ảnh hưởng đến dung lượng và hiệu suất của mô hình.

5. Quá trình Huấn luyện

- **Dataset Preparation:** Quá trình chuẩn bị dữ liệu bao gồm tokenization (ví dụ: sử dụng Byte Pair Encoding - BPE), tạo từ vựng và padding.
- **Loss Function:** Sử dụng cross-entropy loss function để đo lường sự khác biệt giữa phân phối xác suất dự đoán của mô hình và phân phối xác suất thực tế.
- **Optimizer:** Sử dụng thuật toán tối ưu hóa (ví dụ: AdamW) với các tham số như learning rate, weight decay.
- **Learning Rate Schedule:** Sử dụng learning rate schedule (ví dụ: cosine annealing hoặc linear decay) để điều chỉnh learning rate trong quá trình huấn luyện.
- **Batch Size và Gradient Accumulation:** Sử dụng batch size nhỏ và gradient accumulation để huấn luyện mô hình trên phần cứng có bộ nhớ hạn chế.
- **Checkpointing:** Lưu lại checkpoint của mô hình trong quá trình huấn luyện để có thể khôi phục lại trạng thái tốt nhất.
- **Regularization Techniques:** Sử dụng các kỹ thuật regularization như dropout hoặc weight decay để ngăn chặn overfitting.
- **Evaluation Metrics:** Sử dụng perplexity để đánh giá hiệu suất của mô hình.

6. Ứng dụng Thực tế và Ví dụ

NanoGPT có thể được fine-tuned cho nhiều nhiệm vụ NLP cụ thể:

- **Text Classification:** Phân loại văn bản (ví dụ: phân loại cảm xúc).
- **Text Summarization:** Tạo bản tóm tắt ngắn gọn của một văn bản dài.
- **Question Answering:** Trả lời câu hỏi dựa trên một đoạn văn bản đã cho.
- **Code Generation:** Tạo code từ mô tả bằng ngôn ngữ tự nhiên.

Ví dụ Code Fine-tuning (SST2 - Text Classification): (Cần thêm code ví dụ cụ thể cho fine-tuning)

7. So sánh với Các Mô hình Khác

(So sánh nanoGPT với Transformer từ đầu, DistilBERT, MobileBERT, TinyBERT, tập trung vào sự đánh đổi giữa kích thước, hiệu suất, và độ phức tạp).

8. Cải tiến Tiềm năng

(Thảo luận về Quantization, Pruning, Knowledge Distillation, Sparse Attention, Hardware Acceleration).

9. Kết luận

NanoGPT là một công cụ mạnh mẽ để học và thử nghiệm với kiến trúc Transformer. Sự đơn giản, hiệu quả, và linh hoạt của nó làm cho nó trở thành một nguồn tài nguyên vô giá cho giáo dục và nghiên cứu trong NLP. Nó không chỉ là một mô hình nhỏ, mà là một con đường để khám phá và tái hiện nguyên lý hoạt động của các mô hình ngôn ngữ lớn hiện đại.

Key Citations

- [NanoGPT GitHub: The simplest, fastest repository for training/finetuning medium-sized GPTs](#)
- [Training nanoGPT entirely on content from my blog | Simon Willison's TILs](#)
- [Build nanoGPT from scratch: Video+code lecture on building nanoGPT from scratch](#)
- [ng-video-lecture/gpt.py at master · karpathy/ng-video-lecture](#)
- [Train your own language model with nanoGPT | Medium](#)
- [NanoGPT | Hacker News](#)
- [Andrej Karpathy Reproduces GPT-2 in Latest Tutorial](#)